



ELISA

Enabling **Linux** in
Safety Applications

ELISA Workshop London, UK

June 9-11, 2026
Co-hosted with Canonical





ELISA

Enabling **Linux** in
Safety Applications

Defining Linux Kernel Requirements and Test Specifications out of the Kernel Tree

Gabriele Paoloni, Chuck Wolber, Kate Stewart

ELISA Workshop: London, UK

June 9-11, 2026

Co-hosted with Canonical





ELISA

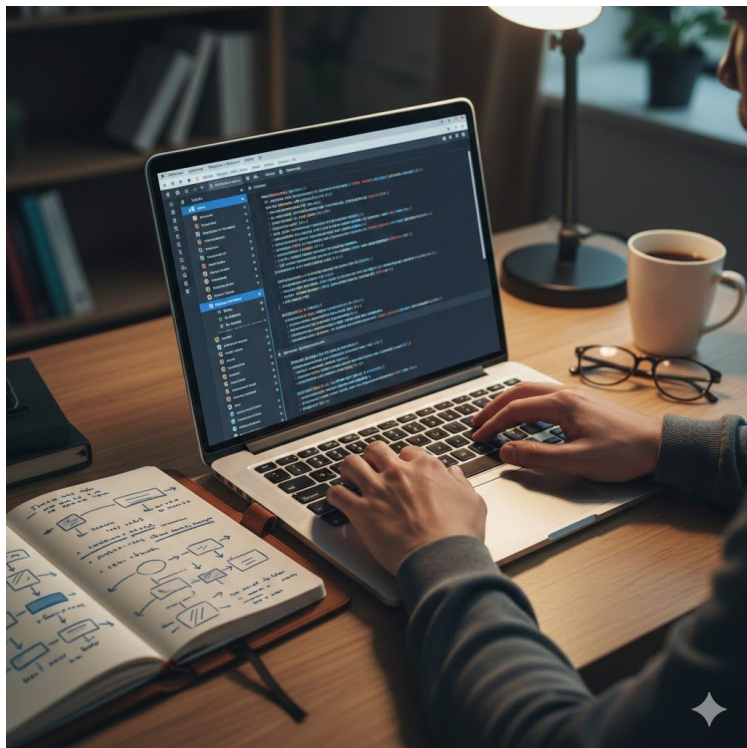
Enabling **Linux** in
Safety Applications

Agenda:

- Introduction to Requirements
- The Initial upstream trials
- The Out-Of-Tree maintenance proposal
- Next Steps

LPC'24: “Defining and maintaining requirements in the Linux Kernel”

Why Documenting the design is important



- New developers and security experts can verify if their proposed modifications are coherent and consistent with the intended behavior (e.g. when modifying the documented symbols).
- Reduction of incorrect or buggy patch proposals by new contributors thanks to faster understanding of the code behavior and side effects
- Testers can test the code against the documented intended behavior and documented constraints of use, hence without the risk of mistaking a bug for a feature. Moreover testing efficiency would be improved by more targeted testing around scope of impact of change.

In general the technical debt associated with Linux would be reduced.

LPC'24: “Improving kernel design documentation and involving experts”

Outcome of the discussion



- Participating maintainers agreed that low level requirements are important to properly uphold intended design;
- Participating maintainers asked to have semantic aspects of the requirements to be added inline with the source code whereas the rest of requirements' formalism should go into sidecar files;
- The requirements format must be compatible with the existing kernel documentation;
- The requirements' formalism should be supported by automation for evaluating requirements against newly submitted code.

The initial requirement trials

```
/**
 * SPDX-Req-ID: [TODO automatically generate it]
 * SPDX-Req-Text:
 * trace_set_clr_event - enable or disable an event within a system.
 * @system: system name (NULL for any system).
 * @event: event name (NULL for all events, within system).
 * @set: 1 to enable, 0 to disable (any other value is invalid).
 *
 * This is a way for other parts of the kernel to enable or disable
 * event recording.
 *
 * Function's expectations:
 * - This function shall retrieve the pointer of the global trace array (global
 *   tracer) and pass it, along the rest of input parameters, to
 *   __ftrace_set_clr_event_nolock.
 * - This function shall properly lock/unlock the global event_mutex
 *   before/after invoking ftrace_set_clr_event_nolock.
 *
 * Returns 0 on success, -ENODEV if the global tracer cannot be retrieved,
 * -EINVAL if the parameters do not match any registered events, any other
 * error condition returned by __ftrace_set_clr_event_nolock.
 */
int trace_set_clr_event(const char *system, const char *event, int set)
{
    struct trace_array *tr = top_trace_array();

    if (!tr)
        return -ENODEV;

    return __ftrace_set_clr_event(tr, NULL, system, event, set, NULL);
}
EXPORT_SYMBOL_GPL(trace_set_clr_event);
```

The template was presented at the 2024 ELISA workshop hosted by NASA.

Positive feedbacks received by Steven Rostedt (TRACING subsystem maintainer).

The main suggestion was: “***before refining the syntax and formalism, work on the semantic aspects and show the value***”

TRACING subsystem's submissions

We mainly worked on
“kernel/trace/trace_events.c” (see [1] and [2])
with **different outcomes**:

a bug was found (see
[3])

From: Steven Rostedt <rostedt@goodmis.org>
To: linux-kernel@vger.kernel.org
Cc: Masami Hiramatsu <mhiramat@kernel.org>,
Mark Rutland <mark.rutland@arm.com>,
Mathieu Desnoyers <mathieu.desnoyers@efficios.com>,
Andrew Morton <akpm@linux-foundation.org>,
Gabriele Paoloni <gpaoloni@redhat.com>
Subject: [for-next][PATCH 08/10] tracing: fix return value in __ftrace_event_enable_disable for TRACE_REG_UNREGISTER
Date: Sun, 23 Mar 2025 08:29:41 -0400 [thread overview]
Message-ID: <20250323122950.397439862@goodmis.org> (raw)
In-Reply-To: 20250323122933.407277911@goodmis.org

From: Gabriele Paoloni <gpaoloni@redhat.com>

When __ftrace_event_enable_disable invokes the class callback to unregister the event, the return value is not reported up to the caller, hence leading to event unregister failures being silently ignored.

This patch assigns the ret variable to the invocation of the event unregister callback, so that its return value is stored and reported to the caller, and it raises a warning in case of error.

Link: <https://lore.kernel.org/20250321170821.101403-1-gpaoloni@redhat.com>
Signed-off-by: Gabriele Paoloni <gpaoloni@redhat.com>
Signed-off-by: Steven Rostedt (Google) <rostedt@goodmis.org>

[1] <https://lore.kernel.org/all/20250814122206.109096-1-gpaoloni@redhat.com/>

[2] <https://lore.kernel.org/linux-trace-kernel/20250814122141.109076-1-gpaoloni@redhat.com/>

[3] <https://lore.kernel.org/all/20250323122950.397439862@goodmis.org/>

TRACING subsystem's submissions

We mainly worked on
“kernel/trace/trace_events.c” (see [1] and [2])
with **different outcomes**:

A redundant flag (SOFT_MODE) was
identified and removed, with the code
reworked and optimised (see [4])

From: Steven Rostedt <rostedt@kernel.org>
To: linux-kernel@vger.kernel.org
Cc: Masami Hiramatsu <mhiramat@kernel.org>,
Mark Rutland <mark.rutland@arm.com>,
Mathieu Desnoyers <mathieu.desnoyers@efficios.com>,
Andrew Morton <akpm@linux-foundation.org>,
Gabriele Paoloni <gpaoloni@redhat.com>
Subject: [\[for-next\] \[PATCH 6/8\] tracing: Remove EVENT_FILE_FL_SOFT_MODE flag](#)
Date: Wed, 23 Jul 2025 10:49:13 -0400 [\[thread overview\]](#)
Message-ID: <20250723144928.341184323@kernel.org> ([raw](#))
In-Reply-To: 20250723144907.219256132@kernel.org

From: Steven Rostedt <rostedt@goodmis.org>

When soft disabling of trace events was first created, it needed to have a way to know if a file had a user that was using it with soft disabled (for triggers that need to enable or disable events from a context that can not really enable or disable the event, it would set SOFT_DISABLED to state it is disabled). The flag SOFT_MODE was used to denote that an event had a user that would enable or disable it via the SOFT_DISABLED flag.

Commit 1cf4c0732db3c ("tracing: Modify soft-mode only if there's no other referrer") fixed a bug where if two users were using the SOFT_DISABLED flag the accounting would get messed up as the SOFT_MODE flag could only handle one user. That commit added the sm_ref counter which kept track of how many users were using the event in "soft mode". This made the SOFT_MODE flag redundant as it should only be set if the sm_ref counter is non zero.

Remove the SOFT_MODE flag and just use the sm_ref counter to know the event is in soft mode or not. This makes the code a bit simpler.

Link: <https://lore.kernel.org/all/20250702111908.03759998@batman.local.home/>

Cc: Masami Hiramatsu <mhiramat@kernel.org>
Cc: Mathieu Desnoyers <mathieu.desnoyers@efficios.com>
Cc: Gabriele Paoloni <gpaoloni@redhat.com>
Link: <https://lore.kernel.org/20250702143657.18dd1882@batman.local.home>
Signed-off-by: Steven Rostedt (Google) <rostedt@goodmis.org>

- [1] <https://lore.kernel.org/all/20250814122206.109096-1-gpaoloni@redhat.com/>
[2] <https://lore.kernel.org/linux-trace-kernel/20250814122141.109076-1-gpaoloni@redhat.com/>
[3] <https://lore.kernel.org/all/20250323122950.397439862@goodmis.org/>
[4] <https://lore.kernel.org/all/20250723144928.341184323@kernel.org/>

TRACING subsystem's submissions

All of lore.kernel.org

search help / color / mirror / Atom feed

From: Gabriele Paoloni <gpaoloni@redhat.com>
To: rosted@goodmis.org, mhiramat@kernel.org,
mathieu.desnoyers@efficios.com, linux-kernel@vger.kernel.org,
linux-trace-kernel@vger.kernel.org
Cc: acarmina@redhat.com, chuck.wolber@boeing.com,
Gabriele Paoloni <gpaoloni@redhat.com>
Subject: [RFC PATCH 1/2] tracing: fixes of ftrace_enable_fops
Date: Thu, 12 Jun 2025 12:43:48 +0200 [thread overview]
Message-ID: <20250612104349.5047-2-gpaoloni@redhat.com> (raw)
In-Reply-To: <20250612104349.5047-1-gpaoloni@redhat.com>

Currently there are different issues associated with ftrace_enable_fops

- event_enable_write: *ppos is increased while not used at all in the write operation itself (following a write, this could lead a read to fail or report a corrupted event status);
- event_enable_read: cnt < strlen(buf) is allowed and this can lead to reading an incomplete event status (i.e. not all status characters are retrieved) and/or reading the status in a non-atomic way (i.e. the status could change between two consecutive reads);
- .llseek is set to default_llseek: this is wrong since for this type of files it does not make sense to reposition the ppos offset. Hence this should be set instead to noop_llseek.

This patch fixes all the issues listed above.

Signed-off-by: Gabriele Paoloni <gpaoloni@redhat.com>
Tested-by: Alessandro Carminati <acarmina@redhat.com>

kernel/trace/trace_events.c | 11 ++++++---
1 file changed, 8 insertions(+), 3 deletions(-)

```
diff --git a/kernel/trace/trace_events.c b/kernel/trace/trace_events.c
index 120531268abf..5e84ef01d0c8 100644
--- a/kernel/trace/trace_events.c
+++ b/kernel/trace/trace_events.c
@@ -1798,6 +1798,13 @@ event_enable_read(struct file *filp, char __user *ubuf, size_t
```

We realised that it is easy to mislead a feature for a bug (see [5]). So we introduced Assumptions of Use (AoUs) to document usage constraints

Just to confirm, I agree with Masami. The enable file is quite special, and I don't see the use of user space playing tricks with it, which even includes lseek. Maybe to keep rewinding a read to get a new status change?

But it usually contains a single character (sometimes two) and a new line. It's not something that's ever been reported as an issue. I rather not touch it if it hasn't been reported as broken because there's some hypothetical use case that can see it as broken.

Documenting its current behavior is perfectly fine with me.

-- Steve

[1] <https://lore.kernel.org/all/20250814122206.109096-1-gpaoloni@redhat.com/>
[2] <https://lore.kernel.org/linux-trace-kernel/20250814122141.109076-1-gpaoloni@redhat.com/>
[3] <https://lore.kernel.org/all/20250323122950.397439862@goodmis.org/>
[4] <https://lore.kernel.org/all/20250723144928.341184323@kernel.org/>
[5] <https://lore.kernel.org/all/20250612104349.5047-2-gpaoloni@redhat.com/>

TRACING subsystem's submissions

```
+/**
+ * event_enable_read - read from a trace event file to retrieve its status.
+ * @filp: file pointer associated with the target trace event.
+ * @ubuf: user space buffer where the event status is copied to.
+ * @cnt: number of bytes to be copied to the user space buffer.
+ * @ppos: the current position in the buffer.
+ *
+ * This is a way for user space executables to retrieve the status of a
+ * specific event
+ *
+ * Function's expectations:
+ * - The global event_mutex shall be taken before performing any operation
+ *   on the target event;
+ *
+ * - The string copied to user space shall be formatted according to the
+ *   status flags from the target event file:
+ *   - If the enable flag is set AND the soft_disable flag is not set then
+ *     the first character shall be set to "1" ELSE it shall be set to "0";
+ *
+ *   - If either the soft_disable flag or the soft_mode flag is set then the
+ *     second character shall be set to "*" ELSE it is skipped;
+ *
+ *   - The string shall be terminated by a newline ("\n") and any remaining
+ *     character shall be set to "0";
+ *
+ * - This function shall invoke simple_read_from_buffer() to perform the copy
+ *   of the kernel space string to ubuf.
+ *
+ * Assumptions of Use:
+ * - The caller shall pass cnt equal or greater than the length of the string
+ *   to be copied to user space;
+ *
+ * - Any read operation on a file descriptor, unless it is the first operation
+ *   following a trace event file open, shall be preceded by an lseek
+ *   invocation to reposition the file offset to zero.
+ *
+ * Context: process context, locks and unlocks event_mutex.
+ *
+ * Return:
+ * * the number of copied bytes on success
+ * * %-ENODEV - the event file cannot be retrieved from the input filp
+ * * any error returned by simple_read_from_buffer
+ */
```

We realised that it is easy to mislead a feature for a bug (see [5]). So we introduced Assumptions of Use (AoUs) to document usage constraints

Just to confirm, I agree with Masami. The enable file is quite special, and I don't see the use of user space playing tricks with it, which even includes lseek. Maybe to keep rewinding a read to get a new status change?

But it usually contains a single character (sometimes two) and a new line. It's not something that's ever been reported as an issue. I rather not touch it if it hasn't been reported as broken because there's some hypothetical use case that can see it as broken.

Documenting its current behavior is perfectly fine with me.

-- Steve

- [1] <https://lore.kernel.org/all/20250814122206.109096-1-gpaoloni@redhat.com/>
[2] <https://lore.kernel.org/linux-trace-kernel/20250814122141.109076-1-gpaoloni@redhat.com/>
[3] <https://lore.kernel.org/all/20250323122950.397439862@goodmis.org/>
[4] <https://lore.kernel.org/all/20250723144928.341184323@kernel.org/>
[5] <https://lore.kernel.org/all/20250612104349.5047-2-gpaoloni@redhat.com/>

/dev/mem submissions

We submitted a patchset with documentations, testable expectations and associated tests (see [7])

However it seems that there is no clarity about why we need this, which APIs will be specified, who is will do the work

[6] <https://lore.kernel.org/all/20250821170419.70668-1-gpaoloni@redhat.com/>

[7] <https://lore.kernel.org/all/20250910170000.6475-1-gpaoloni@redhat.com/>

Subject: [RFC PATCH v2 0/3] Add testable code specifications
Date: Wed, 10 Sep 2025 18:59:57 +0200 [thread overview]
Message-ID: <20250910170000.6475-1-gpaoloni@redhat.com> (raw)

[1] was an initial proposal defining testable code specifications for some functions in /drivers/char/mem.c.

However a Guideline to write such specifications was missing and test cases tracing to such specifications were missing.

This patchset represents a next step and is organised as follows:

- patch 1/3 contains the Guideline for writing code specifications
- patch 2/3 contains examples of code specifications defined for some functions of drivers/char/mem.c
- patch 3/3 contains examples of selftests that map to some code specifications of patch 2/3

[1] <https://lore.kernel.org/all/20250821170419.70668-1-gpaoloni@redhat.com/>

Changes from v1:

- 1) Added a Guideline to write code specifications in the Linux Kernel Documentation
- 2) Addressed Greg KH comments in /drivers/char/mem.c
- 3) Added example of test cases mapping to the code specifications in /drivers/char/mem.c

> 1) In the first part of patch 1/3 we explain why we are doing this and the high > level goals. Do you agree with these? Are these clear?

No, and no.

I think this type of thing is, sadly, folly. You are entering into a path that never ends with no clear goal that you are conveying here to us.

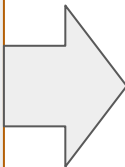
I might be totally wrong, but I fail to see what you want to have happen in the end.

Every in-kernel api documented in a "formal" way like this? Or a subset? If a subset, which ones specifically? How many? And who is going to do that? And who is going to maintain it? And most importantly, why is it needed at all?

For some reason Linux has succeeded in pretty much every place an operating system is needed for cpus that it can run on (zephyr for those others that it can not.) So why are we suddenly now, after many decades, requiring basic user/kernel stuff to be formally documented like this?

/dev/mem submissions

Following LPC25 discussions and feedbacks we proposed a further patchset with the minimum formalism to trace specifications to test cases (see [8])



RFCv1 [1] was an initial proposal defining testable code specifications for some functions in /drivers/char/mem.c. However it was not clear how to write specifications according to the format and wording used for them, plus tests were missing.

RFCv2 [2] proposed a guideline explaining how to write such testable code specifications and also added kselftest for read_mem, however there was reluctance to accept the proposed addition before having clarity on who needs it, who is going to maintain it and for which APIs in the kernel it is needed.

RFCv3 (this one) proposes kernel-doc API documentation according to the current doc-guide for some APIs of /dev/mem introducing the concept of traceable specifications to KUnit tests.

For read_mem() Kunit tests traced to the corresponding specifications are also introduced, so that in case of test failure it is clear which specification is violated and it would be easier to review incoming patches against the intended behavior of the code.



Again this was Rejected.

```
> + * [ID2] - [constraint to be met by the caller]
> + *
> + * [IDn] - [constraint to be met by the caller]
```

So the same "id" is used for a behavior, AND a constraint?

And what defines an "id"? I see in your example you use number.number, but is that specified?

And how is a id going to stay in sync across different files? That feels impossible to maintain for any length of time, and puts a burden on the developer who wishes to add/remove a test or "id", AND a maintainer who has to remember to go and look in multiple places for such an id sync up.

That's just not going to work, sorry.

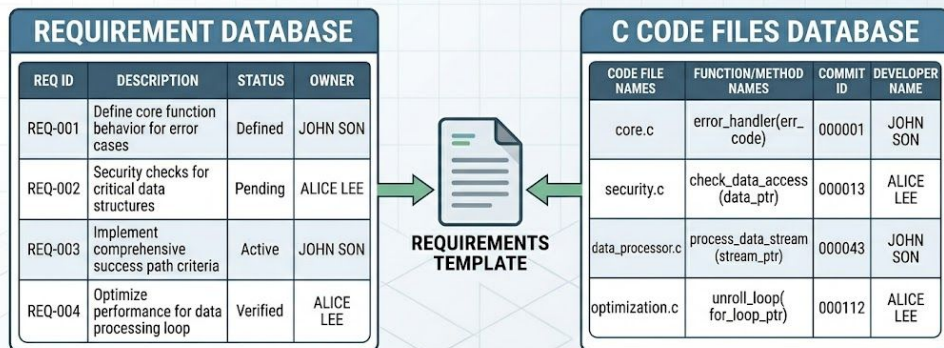
greg k-h

[8] <https://lore.kernel.org/all/20260212124923.222484-1-gpaoloni@redhat.com/>

New Idea: Maintaining Requirements and Test Specs in a separate repo

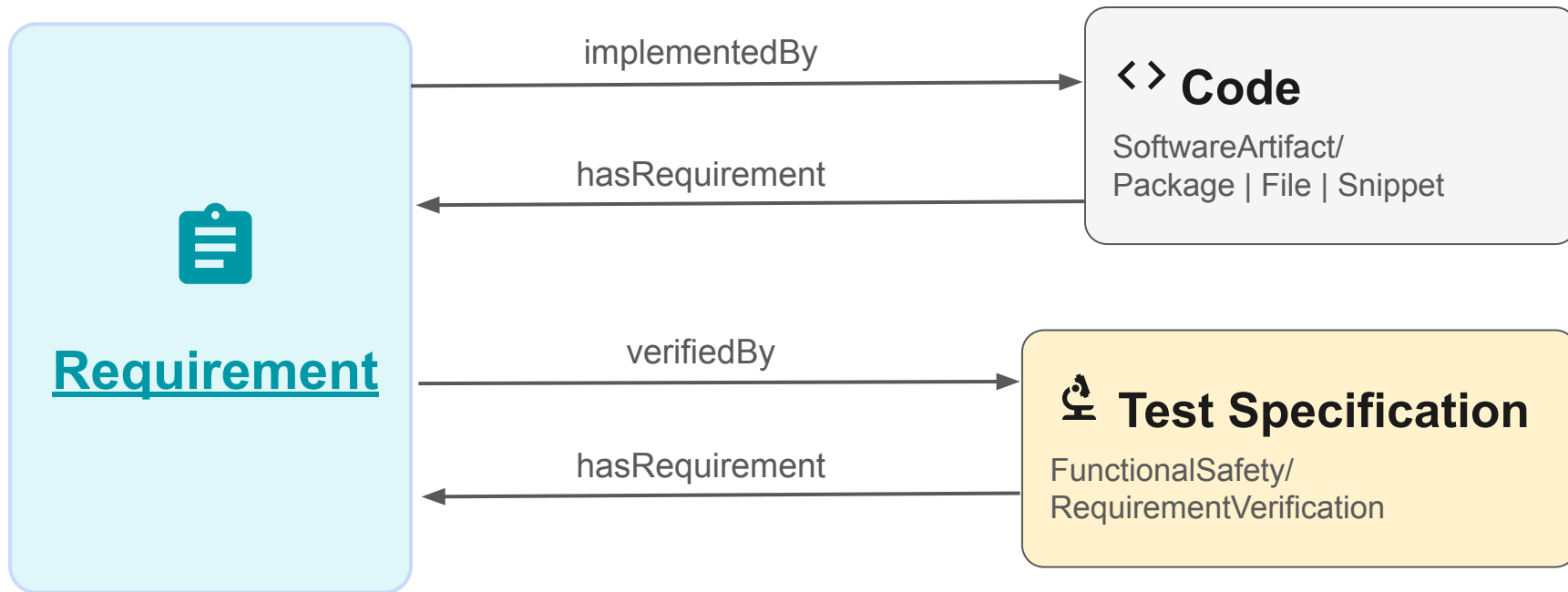
Requirements' Template Goals:

SYSTEM TRACEABILITY: REQUIREMENTS TO CODE



- The syntax should enforce unique IDs, traceability, configuration management, and change management, such that automation stands a high likelihood of generating a complete requirement document tailored to individual needs.
- Should be maintainable together with the code.
- Should support hierarchical traceability to allow refinement of SW dependencies (i.e. cross reference critical APIs or data structures from the template).
- Should describe error conditions and success behavior. This includes cases where success is returned yet behavior diverges from expectations (e.g. cover tricky corner cases).
- Should allow covering both static and dynamic aspects of the code.
- Should support the definition of a test plan (i.e. syntax should enforce testability as well as the avoidance of untestable “shall not” type requirements).
- Should be able to import and export requirements in a format that is compliant with SPDX-3.1

SPDX 3.1-dev (active development)



Requirement (active development)

Field Purpose	Cardinality	SPDX 3.1
Identify Requirement	(1,1)	core/ spdxID core/Requirement/ requirementUID
Requirement Statement	(1,1)	core/Requirement/ requirementStatement
Requirement Integrity	(1,1)	core/requirement/ verifiedUsing (see Integrity Method for options)
Rationale	(0,1)	core/Requirement/ rationale

Under discussion:

requirement state/status - DRAFT | APPROVED | DEPRECATED

Looking to add to relationship as requirements can be reused, evolved, etc. but once created not modified.

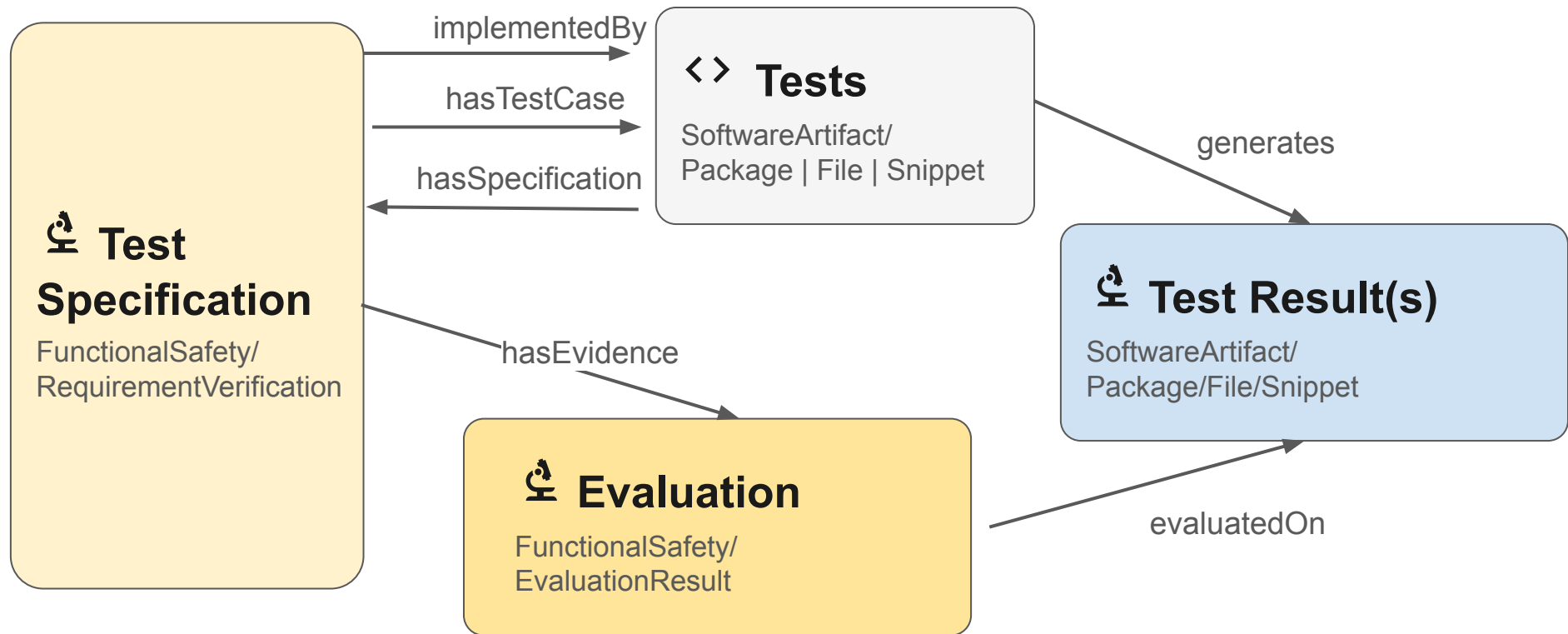
RequirementVerification (active development)

Field Purpose	Cardinality	SPDX 3.1
Identifier of RequirementVerification	(1,1)	core/ spdxID FunctionalSafety/ verificationUID
VerificationMethod	(1,1)	FunctionalSafety/ VerificationType
Why of method selected	(0,1)	core/ Rationale
Any conditions to satisfy after verification performed	(0,1)	FunctionalSafety/ VerificationPostcondition
Any conditions to be satisfied before verification is performed	(0,1)	FunctionalSafety/ VerificationPrecondition

VerificationType is one of **analysis** | **assessment** | **audit** | **demonstration** | **inspection** | **review** | **test** | **other**

Query: should this be renamed? It might be more appropriate to think of this as Test Specification? Verification Specification?

SPDX 3.1-dev (active development)



EvidenceRelationship (active development)

hasEvidence: Every **to** Element is considered as evidence for the **from** Element (**from** hasEvidence **to**).

Field Purpose	Cardinality	SPDX 3.1
Evidence identifier	(1,1)	core/ spdxID FunctionalSafety/ EvidenceUID
Enumeration of types of Evidence Artifacts	(0,*)	FunctionalSafety/ EvidenceType
from	(1,1)	Relationship from RequirementVerification
to	(1,*)	Relationship to objects that provide evidence

EvidenceType is one of **log** | **observation** | **recording** | **report** | **other**

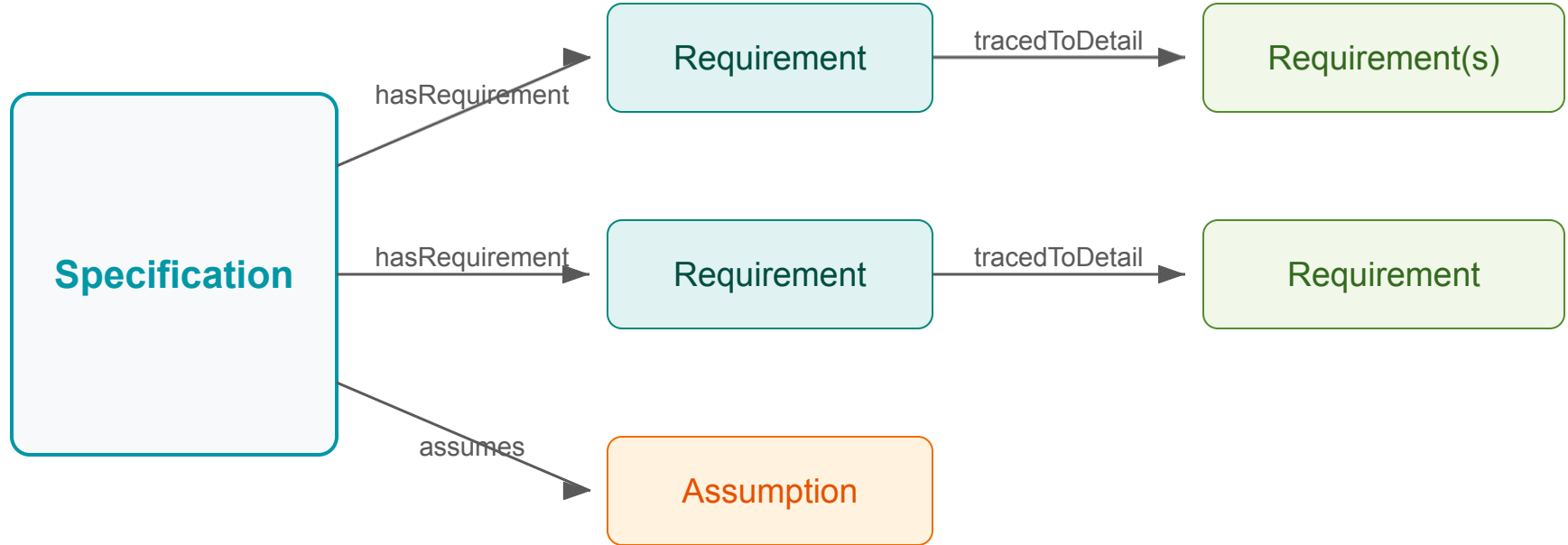
Query: should this be renamed? Restructured? Purpose is to link evidence artifacts to requirementverification

Evaluation Result (active development)

Field Purpose	Cardinality	SPDX 3.1
Evaluation Identifier	(1,1)	core/ spdxID
Result of Evaluation	(1,1)	FunctionalSafety/EvaluationResult/ evaluation which is an evaluationResultType
Rationale	(1,1)	core/element
Evaluation based on	(1,1)	FunctionalSafety/EvaluationResult/evaluationBasedOn which is a RequirementVerification

evaluationResultType is one of **pass** | **fail** | **inconclusive**

SPDX 3.1-dev (active development)



Specification (active development)

Field Purpose	Cardinality	SPDX 3.1
Identify a Specification Artifact	(1,1)	core/ spdxID core/ externalIdentifier
Type of Specification	(0,1)	core/ specType
Name of specification	(0,1)	xsd:string

specificationType is one of **formalStandard** | **regulation** | **specification** | **other**

Assumption (active development)

Field Purpose	Cardinality	SPDX 3.1
Identifier of Assumption	(1,1)	core/ spdxID FunctionalSafety/ assumptionUID
Assumption of use Statement	(1,1)	FunctionalSafety/ assumptionStatement
why of assumption	(0,1)	core/ Rationale
Assumption Integrity	(1,1)	FunctionalSafety/ verifiedUsing (see Integrity Method for options)

Note: assumptions should be able to point to other assumptions via a **"assumes"** relationship.

Next Steps

- 1) Complete the SPDX formalism definition to maintain requirements, evidence, test specifications along the corresponding code and test cases
- 2) Develop tools (e.g. BASIL) to support such a formalism and relationships
- 3) Define a change management process leveraging tools and formalism from points above
- 4) Create repo for requirements and test specifications and start populating it with the Linux kernel data; and work with interested kernel maintainers and KernelCI community to review.
- 5) Spot bugs in existing code and in new contributions, show the value and create an ecosystem of interested parties using all of the above

Licensing of Workshop Results

All work created during the workshop is licensed under Creative Commons Attribution 4.0 International (CC-BY-4.0) [<https://creativecommons.org/licenses/by/4.0/>] by default, or under another suitable open-source license, e.g., GPL-2.0 for kernel code contributions.

You are free to:

- Share — copy and redistribute the material in any medium or format
- Adapt — remix, transform, and build upon the material for any purpose, even commercially.

The licensor cannot revoke these freedoms as long as you follow the license terms.

Under the following terms:

Attribution — You must give appropriate credit, provide a link to the license, and indicate if changes were made. You may do so in any reasonable manner, but not in any way that suggests the licensor endorses you or your use.

No additional restrictions — You may not apply legal terms or technological measures that legally restrict others from doing anything the license permits.



ELISA
Enabling **Linux** in
Safety Applications



WORKSHOP

